

Module 3: Data Organization and Manipulation using Collections

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai campus

premanand.s@vit.ac.in

September 8, 2025

In module 3,

- Data Handling Strategies Using Lists, Tuples, Dictionary and Set
- Data Comprehension
- Data Modification
- Grouping and Categorization
- Iterators
- Data Selection - Ordered, Unordered and Unique Data
- Searching Techniques: Linear Search, Binary Search
- Sorting Techniques: Bubble Sort, Selection Sort- Insertion Sort- Quick Sort- Merge Sort.

What is a Dictionary?

- Stores data in **key-value pairs**.
- Keys are unique and immutable.
- Values can be any datatype.
- Fast lookup using hash table.

Example:

```
my_dict = {key1: value1 , key2: value2 , ...}
```

```
student = {"name": "Python", "age": 21, "marks": 85}  
print(student)  
type(student)
```

Why is Dictionary Different from Other Data Types?

- List/Tuple/Set → store single values at each position.
- Dictionary → stores data as key: value pairs.
- Keys are unique immutable (string, number, tuple).
- Values can be any type (list, dict, set, etc.).

Example:

Word = Key

Meaning = Value

Characteristics of Dictionary

- Unordered (before Python 3.7 → order not guaranteed).
- Mutable → values can be updated, new pairs can be added/removed.
- Keys must be unique.
- Fast lookup (hash table-based).

Creating Dictionary

Example:

```
student = {"name": "Python", "age": 21}
```

```
person = dict(name="Bob", city="London")
```

```
pairs = [("a", 1), ("b", 2)]
```

```
d = dict(pairs)
```

```
print(d)
```

```
keys = ["a", "b", "c"]
```

```
d = dict.fromkeys(keys, 0)
```

```
print(d)
```

Accessing Elements

Example:

```
student = {"name": "Python", "age": 21}
print(student["name"])    # using keys

print(student.get("marks", "Not Available")) #using get()
```

Updating and Adding Data

Example:

```
student = {"name": "Python", "age": 21}
```

```
student["age"] = 22
```

```
student["marks"] = 90
```

```
print(student)
```


Removing Data

Example:

```
student = {"name": "Python", "age": 21}
```

```
student.pop("marks")
```

```
student.popitem()
```

```
del student["age"]
```

```
student.clear()
```

Dictionary Methods

```
d = {"a": 1, "b": 2, "c": 3}
print(d.keys())
print(d.values())
print(d.items())
```

```
for k, v in student.items(): #iteration
    print(k, "—>", v)
```

```
d.update({"d": 4, "a": 5})
print(d) # from another dictionary
```

```
d2 = d.copy() #shallow copy
```

```
d.setdefault("e", 10) # adds key only if not present
print(d)
```

Dictionary Methods

```
students = {  
    "s1": {"name": "Prem", "marks": 90},  
    "s2": {"name": "Anand", "marks": 85}  
}  
print(students["s1"]["name"])
```

```
d1 = {"a":1, "b":2}  
d2 = {"b":3, "c":4}  
merged = d1 | d2 #merging  
print(merged)
```

Dictionary Comprehension

```
{key_expression: value_expression for  
    item in iterable if condition}
```

```
squares = {x: x**2 for x in range(5)}  
print(squares)  #squaring of numbers
```

```
words = ["apple", "banana", "cherry"]  
lengths = {w: len(w) for w in words}  
print(lengths)  #mapping
```

Dictionary Comprehension

```
even_squares = {x: x**2 for x in  
                 range(10) if x % 2 == 0}  
print(even_squares)
```

Dictionary Comprehension

```
matrix = [[1,2,3],[4,5,6]]  
dict_matrix = {i:{j: val for j, val in enumerate(row)}  
               for i, row in enumerate(matrix)}  
print(dict_matrix)
```

Dictionary Comprehension

```
celsius = [0, 10, 20, 30]
fahrenheit = {c: (9/5)*c + 32 for c in celsius}
print(fahrenheit)
```

Questions

- Create a dictionary of 5 fruits with their colors.
- Access the value of a specific key using both `[]` and `get()`.
- Add a new key-value pair to a dictionary and print it.
- Remove an item from a dictionary using `pop()` and `del`.

Questions

- Write a Python program to count word frequency in a sentence.
- Merge two dictionaries (d1 and d2) without using loops.
- Create a dictionary from two lists: names = ["Alice", "Bob", "Charlie"], marks = [85, 92, 78]
- Iterate through a dictionary and print keys where values are greater than 80.

Questions

- Write a program to create a nested dictionary for storing student details (name, age, marks) and access marks of a specific student.
- Using dictionary comprehension, create a dictionary of numbers from 1 to 10 with their cubes, but only include odd numbers.
- Given a string, build a dictionary that counts frequency of each character and print the most frequent character.